Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO



2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



14

Autenticação

Aula 53 a 56 - Desenvolvimento de Sistemas
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI



14.1

Criando branches

Organizando o projeto em branches

Firjan SENAI

Por que criar uma nova branch?

Antes de iniciar a autenticação, vamos preservar uma versão do projeto exatamente como ele está agora.

Assim, teremos uma branch com tudo o que foi desenvolvido até aqui e outra branch onde serão implementadas as novas funcionalidades de autenticação.

Saiba Mais!



No Git, uma branch funciona como uma linha de desenvolvimento separada.
Isso permite criar novas funcionalidades sem alterar imediatamente o código principal do projeto.

Neste momento, teremos:

- main → versão principal do projeto;
- antes-autenticacao → cópia do projeto exatamente como está agora;
- autenticacao → branch onde desenvolveremos as novas funcionalidades.

Organizando projeto com Git

Criando branches

Abra o git bash
Navegue até a pasta do seu projeto
Execute os comandos:

- git switch main
- git branch antes-autenticação
- git switch –c autenticacao

Para conferir as branches:

- git branch

O resultado será semelhante a:

- git branch

```
$ git branch
  antes-autenticacao
* autenticacao
  main
```

Observação importante: depois disso, todo o desenvolvimento da autenticação será feito na branch autenticacao. A branch antes-autenticacao não deverá ser alterada.


```
MINGW64:/c/Users/vitor/OneDrive - Firjan/Turmas/Técnico em Desenvolvimento de Sistemas/Específico Profissiona...
vitor@DESKTOP-PJHOFLD MINGW64 ~/OneDrive - Firjan/Turmas/Técnico
 em Desenvolvimento de Sistemas/Específico Profissional 2/Desenv
olvimento de Sistemas/Recursos/teste (main)
$ git switch main
Already on 'main'
Your branch is up to date with 'origin/main'.

vitor@DESKTOP-PJHOFLD MINGW64 ~/OneDrive - Firjan/Turmas/Técnico
 em Desenvolvimento de Sistemas/Específico Profissional 2/Desenv
olvimento de Sistemas/Recursos/teste (main)
$ git branch antes-autenticacao

vitor@DESKTOP-PJHOFLD MINGW64 ~/OneDrive - Firjan/Turmas/Técnico
 em Desenvolvimento de Sistemas/Específico Profissional 2/Desenv
olvimento de Sistemas/Recursos/teste (main)
$ git switch -c autenticacao
Switched to a new branch 'autenticacao'

vitor@DESKTOP-PJHOFLD MINGW64 ~/OneDrive - Firjan/Turmas/Técnico
 em Desenvolvimento de Sistemas/Específico Profissional 2/Desenv
olvimento de Sistemas/Recursos/teste (autenticacao)
$ git branch
  antes-autenticacao
* autenticacao
  main

vitor@DESKTOP-PJHOFLD MINGW64 ~/OneDrive - Firjan/Turmas/Técnico
 em Desenvolvimento de Sistemas/Específico Profissional 2/Desenv
olvimento de Sistemas/Recursos/teste (autenticacao)
$
```



14.1

Spring Security e Resource Server

Dependência necessárias para autenticação

Firjan SENAI

Firjan SENAI

Spring Security

O que é o Spring Security?

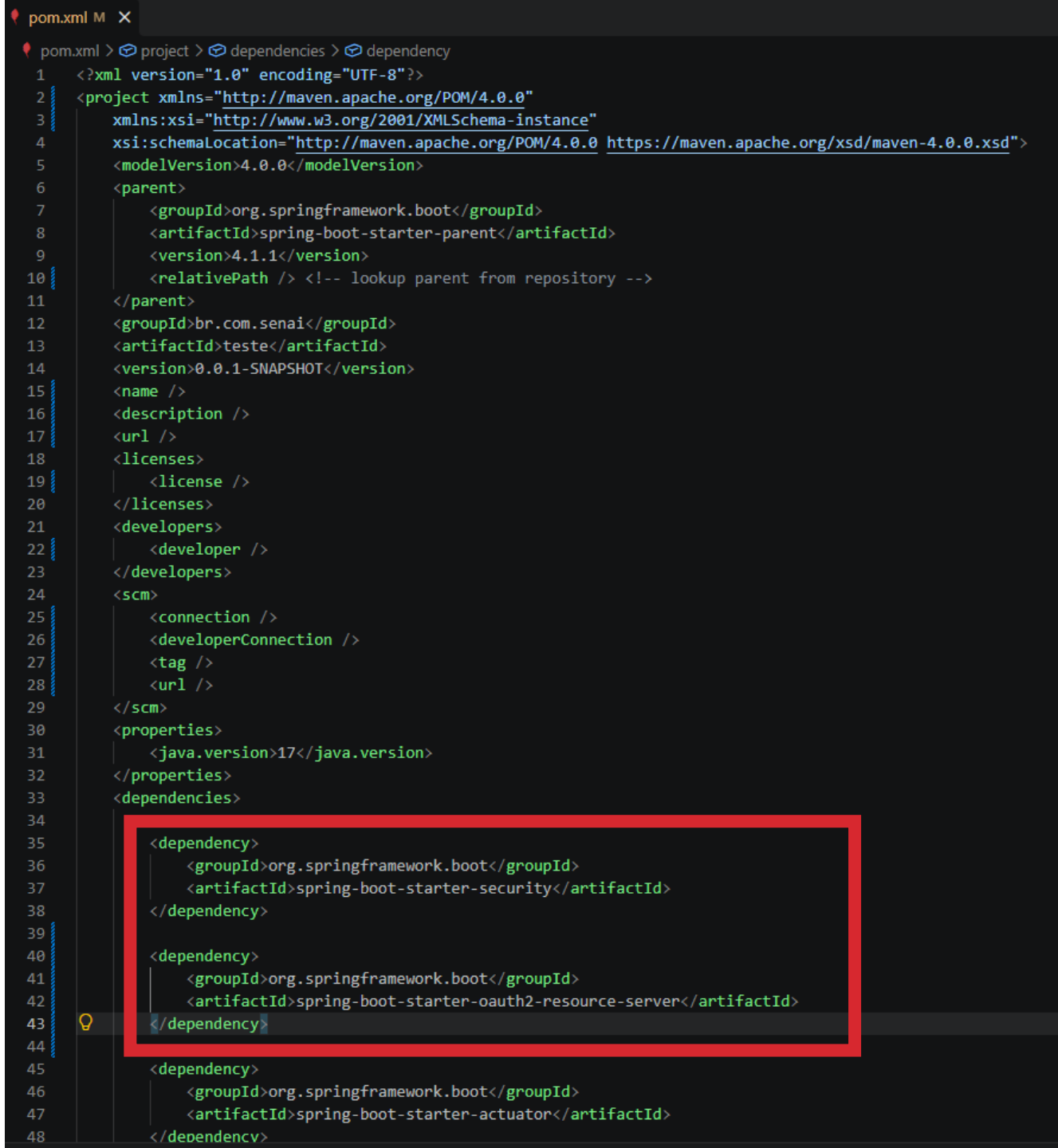
O **Spring Security** é uma ferramenta do ecossistema Spring utilizada para proteger uma aplicação.

Ele permite controlar **quem pode acessar o sistema** e **quais rotas cada usuário pode utilizar**.

Com ele, podemos implementar recursos como:

- autenticação de usuário e senha;
- proteção das rotas da API;
- controle de acesso;
- definição de rotas públicas e privadas.

No arquivo pom.xml, mantenha o Spring Security e adicione o Resource Server. Esse segundo starter fornece os componentes utilizados para validar requisições que enviam um Bearer Token no formato JWT.



Rotas protegidas pelo Spring Security

Ao adicionar a dependência do Spring Security, todas as rotas da aplicação passam a exigir autenticação automaticamente.

Atenção



Ao adicionar o Spring Security ao projeto, ele passa a proteger a aplicação automaticamente.

Por isso, nas próximas etapas será necessário configurar quais rotas poderão ser acessadas livremente e quais exigirão autenticação.

Autenticação com JWT

JSON Web Token

Saiba Mais!



JWT e um formato de token que pode representar a identidade de um usuário. Depois que o login e a senha forem conferidos, o backend gera um token assinado e o devolve ao cliente.

Nas requisições seguintes, o cliente envia o token no lugar da senha. O backend valida sua assinatura e seu prazo de expiração antes de liberar o acesso.

Resumo



O usuário envia a senha no login. Depois disso, as rotas protegidas recebem somente o token no cabeçalho Authorization.

JWT e segurança da conexão

JWT não criptografa a conexão e não torna o HTTP seguro. Se o token for capturado, outra pessoa podera utiliza-lo enquanto ele estiver valido. Em produção, a API deve utilizar HTTPS.

Atenção



O JWT evita o envio repetido da senha, mas o token também e uma credencial. Ele deve ser protegido, possuir expiração e nunca deve conter senha ou outros dados secretos.

O conteúdo do token pode ser lido. A assinatura serve para permitir que o backend detecte alterações no token.

Fluxo de autenticação com Spring Boot + BCrypt + JWT

Visão geral do passo a passo

1 Cadastro do usuário

O usuário é cadastrado e a senha é transformada em hash BCrypt antes de ser salva no banco.

`senha: 123456` → `$2a$10$...`

2 Envio do login

O cliente envia login e senha para a rota /auth/login.

```
POST /auth/login
{
  "login": "usuario",
  "senha": "123456"
}
```

3 Verificação da senha

O Spring consulta o banco e compara a senha recebida com o hash BCrypt salvo.



Banco de dados

BCrypt compara a senha

4 Geração do JWT

Se os dados estiverem corretos, a API gera um JWT assinado e com prazo de expiração.

`eyJ...` . `abc...` . `xyz...`

JWT (3 partes)

`exp: 1712345678`

5 Acesso às rotas protegidas

O cliente envia o Bearer Token nas requisições para rotas protegidas.

```
GET /api/produtos
Authorization: Bearer eyJ...
```

6 Validação do token

Antes de liberar o controller, o Spring valida assinatura, expiração e dados do token.

- assinatura
- expiração
- claims

Usuários da aplicação

Criando a tabela de usuários

No pacote model, crie o arquivo:

- Usuario.java

O atributo unique = true impede que dois usuários tenham o mesmo login.

Repositório de usuários

Durante a autenticação, o Spring Security precisará consultar o banco de dados para verificar se o login informado existe.

No pacote repository, crie o arquivo:

- UsuarioRepository.java

*O método findByLogin(String login) será interpretado automaticamente pelo Spring Data JPA como uma consulta semelhante a SELECT * FROM usuario WHERE login = ?;*

Usuario.java

```java
package br.com.senai.teste.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "usuario")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true)
    private String login;

    @Column(nullable = false)
    private String senha;

    public Usuario() {
    }

    public Usuario(String login, String senha) {
        this.login = login;
        this.senha = senha;
    }

    public Integer getId() {
        return id;
    }

    public String getLogin() {
        return login;
    }

    public void setLogin(String login) {
        this.login = login;
    }

    public String getSenha() {
        return senha;
    }

    public void setSenha(String senha) {
        this.senha = senha;
    }
}
```

UsuarioRepository.java

```java
package br.com.senai.teste.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import br.com.senai.teste.model.Usuario;

public interface UsuarioRepository
        extends JpaRepository<Usuario, Integer> {

    Show AOT-generated Implementation, Query, etc...
    Optional<Usuario> findByLogin(String login);
}
```

Dados para cadastrar um usuário

Criando o DTO de cadastro

Não utilizaremos o Usuario diretamente no corpo da requisição. O cadastro receberá somente os dados permitidos por meio de um DTO.

No pacote dto, crie o arquivo:

- UsuarioRequest.java

As validações impedem o cadastro com login ou senha vazios. A senha também deverá possuir pelo menos seis caracteres.

Neste momento, a senha ainda chega ao backend como texto comum.

UsuarioRequest.java U

src > main > java > br > com > senai > teste > dto > UsuarioRequest.java > UsuarioRequest

```java
package br.com.senai.teste.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class UsuarioRequest {

    @NotBlank(message = "O login é obrigatório")
    private String login;

    @NotBlank(message = "A senha é obrigatória")
    @Size(min = 6, message = "A senha deve possuir pelo menos 6 caracteres")
    private String senha;

    public String getLogin() {
        return login;
    }

    public void setLogin(String login) {
        this.login = login;
    }

    public String getSenha() {
        return senha;
    }

    public void setSenha(String senha) {
        this.senha = senha;
    }
}
```

Firjan SENAI

Protegendo as senhas

Configurando o BCrypt

O sistema não deve armazenar a senha exatamente como foi enviada pelo usuário. Utilizaremos o BCryptPasswordEncoder para gerar um hash antes de salvar o cadastro.

Crie o pacote:

- Config

Dentro dele, crie o arquivo:

- SecurityConfig.java

O @Bean registra o objeto criado pelo método dentro do Spring. Assim, outras classes poderão solicitar um PasswordEncoder pelo construtor.

Quando receber uma senha como:

- senai123

o BCrypt poderá gerar um resultado semelhante a:

- $2a$10$Ru3S8QyV7bF0R3m...

```
SecurityConfig.java U
src > main > java > br > com > senai > teste > config > SecurityConfig.java > Language Support for Java(TM) by
1  package br.com.senai.teste.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.http.HttpMethod;
6  import org.springframework.security.config.Customizer;
7  import org.springframework.security.config.annotation.web.builders.HttpSecurity;
8  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
9  import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.web.SecurityFilterChain;
11
12 @Configuration
13 public class SecurityConfig {
14
15     @Bean
16     public PasswordEncoder passwordEncoder() {
17         return new BCryptPasswordEncoder();
18     }
19
20     @Bean
21     public SecurityFilterChain securityFilterChain(
22             HttpSecurity http) throws Exception {
23
24         http
25             .csrf(csrf -> csrf.disable())
26             .authorizeHttpRequests(auth -> auth
27                 .requestMatchers(
28                     HttpMethod.POST,
29                     "/usuarios"
30                 ).permitAll()
31                 .anyRequest().authenticated()
32             )
33             .httpBasic(Customizer.withDefaults());
34
35         return http.build();
36     }
37 }
```

Serviço de usuários

Protegendo a senha antes de salvar

No pacote service, crie o arquivo:

- UsuarioService.java

O método encode() transforma a senha antes que ela seja enviada ao repositório

O sistema também consulta o banco antes do cadastro. Caso o login já exista, a API responderá: 409 Conflict

A senha original será utilizada somente durante a requisição. No banco de dados, ficará armazenado apenas o hash gerado pelo BCrypt.

UsuarioService.java U

src > main > java > br > com > senai > teste > service > UsuarioService.java > Language Support for Java

```java
package br.com.senai.teste.service;

import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

import br.com.senai.teste.dto.UsuarioRequest;
import br.com.senai.teste.model.Usuario;
import br.com.senai.teste.repository.UsuarioRepository;

@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;

    public UsuarioService(
            UsuarioRepository usuarioRepository,
            PasswordEncoder passwordEncoder) {

        this.usuarioRepository = usuarioRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public Usuario cadastrar(UsuarioRequest dados) {

        if (usuarioRepository.findByLogin(dados.getLogin()).isPresent()) {
            throw new ResponseStatusException(
                    HttpStatus.CONFLICT,
                    "Este login já está cadastrado"
            );
        }

        String senhaProtegida =
                passwordEncoder.encode(dados.getSenha());

        Usuario usuario = new Usuario(
                dados.getLogin(),
                senhaProtegida
        );

        return usuarioRepository.save(usuario);
    }
}
```

Firjan SENAI

Controller de usuários

Criando a rota de cadastro

No pacote controller, crie o arquivo:

- UsuarioController.java

A rota criada será:

- POST /usuarios

O corpo da requisição deverá ser:

-
```
{
  "login": "maria",
  "senha": "senai123"
}
```

O controller devolve diretamente um objeto Usuario. Por isso, a resposta apresentará também o hash da senha:

-
```
{
  "id": 1,
  "login": "maria",
  "senha": "$2a$10$..."
}
```

UsuarioController.java U

src > main > java > br > com > senai > teste > controller > UsuarioController.java > Lang

```java
package br.com.senai.teste.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.senai.teste.dto.UsuarioRequest;
import br.com.senai.teste.model.Usuario;
import br.com.senai.teste.service.UsuarioService;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @PostMapping
    public ResponseEntity<Usuario> cadastrar(
            @Valid @RequestBody UsuarioRequest dados) {

        Usuario usuarioCadastrado =
                usuarioService.cadastrar(dados);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(usuarioCadastrado);
    }
}
```

Resposta segura do cadastro

Evitando o retorno da senha

O objeto Usuario possui o campo senha. Se ele for devolvido diretamente pelo controller, o hash poderá aparecer no JSON da resposta.

No pacote dto, crie o arquivo:

- UsuarioResponse.java

Esse DTO possui somente os dados que poderão ser apresentados ao cliente:

- Id
- login

O campo senha não será enviado, nem mesmo em sua forma protegida.

O hash precisa permanecer armazenado somente no banco, pois será utilizado internamente pelo Spring Security para conferir a senha durante a autenticação.

UsuarioResponse.java U

src > main > java > br > com > senai > teste > dto > UsuarioResponse.jav

```java
package br.com.senai.teste.dto;

import br.com.senai.teste.model.Usuario;

public class UsuarioResponse {

    private Integer id;
    private String login;

    public UsuarioResponse(Usuario usuario) {
        this.id = usuario.getId();
        this.login = usuario.getLogin();
    }

    public Integer getId() {
        return id;
    }

    public String getLogin() {
        return login;
    }
}
```

Firjan SENAI

Aplicando a resposta segura

Alterando o retorno do controller

No arquivo UsuarioController.java:

- altere o método cadastrar
- Adicione a importação

Agora, o controller ainda recebe os dados pelo UsuarioRequest, mas devolve um UsuarioResponse.

Teste novamente:

- POST /usuários
-

```
{
  "login": "joao",
  "senha": "senai123"
}
```

A resposta deverá apresentar apenas:

-

```
{
  "id": 2,
  "login": "joao"
}
```

UsuarioController.java U

src > main > java > br > com > senai > teste > controller > UsuarioController.java > ...

```java
package br.com.senai.teste.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.senai.teste.dto.UsuarioRequest;
import br.com.senai.teste.dto.UsuarioResponse;
import br.com.senai.teste.model.Usuario;
import br.com.senai.teste.service.UsuarioService;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @PostMapping
    public ResponseEntity<UsuarioResponse> cadastrar(
            @Valid @RequestBody UsuarioRequest dados) {

        Usuario usuarioCadastrado =
                usuarioService.cadastrar(dados);

        UsuarioResponse usuarioResponse = new UsuarioResponse(usuarioCadastrado);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(usuarioResponse);
    }
}

```

Firjan SENAI

Autenticação pelo banco de dados

Localizando o usuário durante o login

No pacote service, crie o arquivo:

- AutenticacaoService.java
- Adicione a importação

A interface UserDetailsService representa o serviço que o Spring Security utiliza para procurar um usuário durante a autenticação.

O método loadUserByUsername(String login) é executado automaticamente quando uma requisição envia login e senha.

Se o login existir, o método devolve os dados necessários para o Spring comparar a senha recebida com o hash armazenado.

Se o login não existir, será lançada uma UsernameNotFoundException e a autenticação será recusada.

A função .roles("USER") atribui temporariamente o perfil USER a todos os usuários. Neste momento, o perfil ainda não será armazenado no banco.

AutenticacaoService.java U

src > main > java > br > com > senai > teste > service > AutenticacaoService.java > AutenticacaoService > load

```java
package br.com.senai.teste.service;

import java.util.Optional;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import br.com.senai.teste.model.Usuario;
import br.com.senai.teste.repository.UsuarioRepository;

@Service
public class AutenticacaoService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public AutenticacaoService(
            UsuarioRepository usuarioRepository) {

        this.usuarioRepository = usuarioRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String login)
            throws UsernameNotFoundException {

        Optional<Usuario> usuarioEncontrado = usuarioRepository.findByLogin(login);

        if (usuarioEncontrado.isEmpty()) {
            throw new UsernameNotFoundException(
                    "Usuário não encontrado");
        }

        Usuario usuario = usuarioEncontrado.get();

        return User
                .withUsername(usuario.getLogin())
                .password(usuario.getSenha())
                .roles("USER")
                .build();
    }
}
```

Ln 29,

Requisição de login

Criando o LoginRequest

Representa o JSON enviado quando o usuário deseja entrar no sistema.

No pacote dto, crie o arquivo:

- LoginRequest.java

O login e a senha serão usados apenas para essa verificação. Quando estiverem corretos, a resposta apresentara um JWT.

LoginRequest.java U

src > main > java > br > com > senai > teste > dto > LoginRequest.java > { } br.c

```java
package br.com.senai.teste.dto;

import jakarta.validation.constraints.NotBlank;

public class LoginRequest {

    @NotBlank(message = "O login e obrigatorio")
    private String login;

    @NotBlank(message = "A senha e obrigatoria")
    private String senha;

    public String getLogin() {
        return login;
    }

    public void setLogin(String login) {
        this.login = login;
    }

    public String getSenha() {
        return senha;
    }

    public void setSenha(String senha) {
        this.senha = senha;
    }
}
```

Resposta da autenticação

Criando o TokenResponse

Esse DTO evita devolver apenas uma String solta e deixa claro qual informação foi produzida pelo login.

No pacote dto, crie o arquivo:

- TokenResponse.java

O cliente devera guardar o token e envia-lo nas requisições seguintes.

Exemplo de JSON:

-
```
{
  "token": "eyJhbGciOiJIUzI1NiJ9..."
}
```

```java
package br.com.senai.teste.dto;

public class TokenResponse {

    private String token;

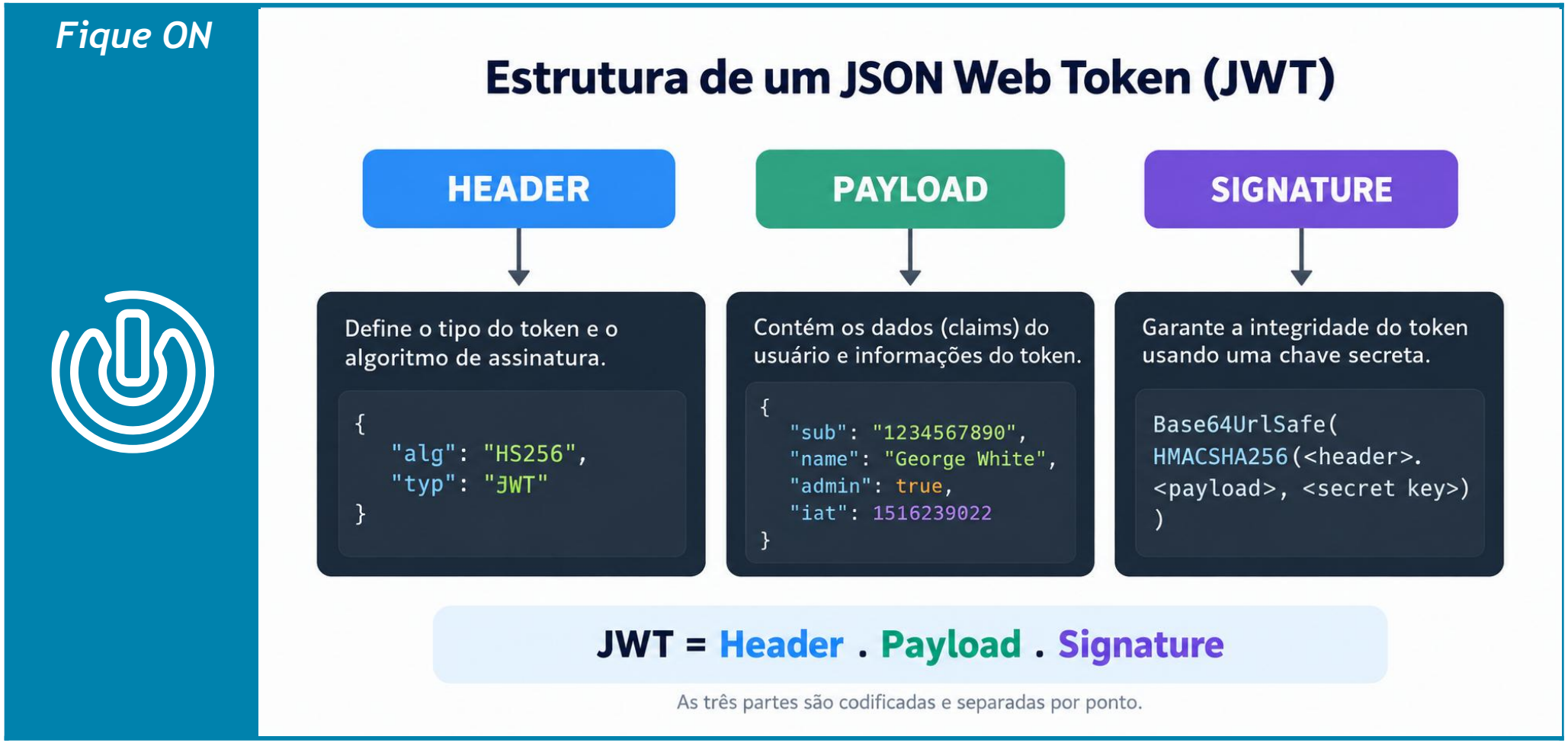
    public TokenResponse(String token) {
        this.token = token;
    }

    public String getToken() {
        return token;
    }
}
```

JWT

Entendendo o formato do token JWT





Assinando e validando o JWT

Configurando a chave secreta

No application.properties, adicione uma chave com pelo menos 32 caracteres:

- jwt.secret=chave-jwt-local

O JwtEncoder usa essa chave para assinar os tokens. O JwtDecoder usa a mesma chave para verificar se o token e autentico e se nao foi alterado.

Esta chave fixa serve para o ambiente local da aula. Em producao, ela deve ficar em uma variavel de ambiente ou em um gerenciador de segredos e nunca deve ser publicada no GitHub.

application.properties M

src > main > resources > application.properties

```
1  spring.application.name=teste
2
3
4  spring.datasource.url=jdbc:mysql://localhost:3306/senai
5  spring.datasource.username=root
6  spring.datasource.password=123.qwe
7
8  spring.jpa.hibernate.ddl-auto=update
9  spring.jpa.show-sql=true
10
11 jwt.secret=chave-jwt-local-com-pelo-menos-32-caracteres
12
13 server.port=9090
```

Porque 32 caracteres?

Porque estamos utilizando o algoritmo HS256.
Ele trabalha com uma chave de, no mínimo, 256 bits: 256 bits ÷ 8 = 32 bytes

A especificação do JWT exige uma chave de pelo menos 256 bits para o HS256. Uma chave menor reduziria a segurança da assinatura.

Saiba Mais! 	Bytes ou caracteres? O correto é dizer 32 bytes, não necessariamente 32 caracteres. Com caracteres ASCII comuns, cada caractere ocupa um byte. caracteres = 32 bytes = 256 bits
Fique ON 	Para gerar essa chave podemos utilizar ferramentas pesquisando ou até mesmo IA. Mas é recomendado gerar a chave e adicionar alguns caracteres próprios depois para garantir ainda mais segurança. https://jwtsecretkeygenerator.com/pt/

Firjan SENAI

Encoder e decoder do token

Preparando o SecurityConfig

No pacote config, crie SecurityConfig.java com as configurações ao lado.

O SecretKey representa a chave utilizada pelo algoritmo HS256.

O JwtEncoder cria e assina o JWT durante o login. O JwtDecoder valida assinatura, emissor e expiração quando uma rota protegida recebe o token.

Esses objetos são registrados com @Bean para que o Spring possa utiliza-los em outras classes.

```java
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {…

    @Bean
    public JwtEncoder jwtEncoder(SecretKey secretKey) {

        OctetSequenceKey jwtKey = new OctetSequenceKey.Builder(secretKey)
                .algorithm(JWSAlgorithm.HS256)
                .build();

        JWKSource<SecurityContext> jwkSource = new ImmutableJWKSet<>(new JWKSet(jwtKey));

        return new NimbusJwtEncoder(jwkSource);
    }

    @Bean
    public JwtDecoder jwtDecoder(SecretKey secretKey) {

        NimbusJwtDecoder decoder = NimbusJwtDecoder
                .withSecretKey(secretKey)
                .macAlgorithm(MacAlgorithm.HS256)
                .build();

        OAuth2TokenValidator<Jwt> validador = JwtValidators.createDefaultWithIssuer("teste-api");

        decoder.setJwtValidator(validador);

        return decoder;
    }

    @Bean
    public SecretKey jwtSecretKey(
            @Value("${jwt.secret}") String secret) {

        byte[] chave = secret.getBytes(StandardCharsets.UTF_8);

        if (chave.length < 32) {
            throw new IllegalArgumentException(
                    "A chave JWT deve possuir pelo menos 32 caracteres");
        }

        return new SecretKeySpec(chave, "HmacSHA256");
    }

    @Bean
    public AuthenticationManager authenticationManager(…

    @Bean
    public SecurityFilterChain securityFilterChain(…
}
```

Firjan SENAI

AuthenticationManager

Preparando o SecurityConfig

Ainda no arquivo SecurityConfig.java adicione o método:

- authenticationManager que recebe login e senha.

```java
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {…

    @Bean
    public JwtEncoder jwtEncoder(SecretKey secretKey) {…

    @Bean
    public JwtDecoder jwtDecoder(SecretKey secretKey) {…

    @Bean
    public SecretKey jwtSecretKey(…

    @Bean
    public AuthenticationManager authenticationManager(
                    AuthenticationConfiguration configuration)
                    throws Exception {

        return configuration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(…
}
```

Servico de token

Criando o TokenService

No pacote service, crie TokenService.java. O método gerarToken() monta as informações do JWT e solicita sua assinatura ao JwtEncoder.

O token conterá:

- iss: aplicação que emitiu o token;
- sub: login do usuário;
- iat: momento da emissão;
- exp: momento da expiração.

Neste exemplo, o token permanecera valido por uma hora.

TokenService.java U

src > main > java > br > com > senai > teste > service > TokenService.java > { } br.com.senai.teste

```java
package br.com.senai.teste.service;

import java.time.Instant;

import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtClaimsSet;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtEncoderParameters;
import org.springframework.security.oauth2.jwt.JwsHeader;
import org.springframework.stereotype.Service;

@Service
public class TokenService {

    private final JwtEncoder jwtEncoder;

    public TokenService(JwtEncoder jwtEncoder) {
        this.jwtEncoder = jwtEncoder;
    }

    public String gerarToken(String login) {

        Instant agora = Instant.now();
        Instant expiracao = agora.plusSeconds(3600);

        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer("teste-api")
                .subject(login)
                .issuedAt(agora)
                .expiresAt(expiracao)
                .build();

        JwsHeader header = JwsHeader
                .with(MacAlgorithm.HS256)
                .build();

        JwtEncoderParameters parametros =
                JwtEncoderParameters.from(header, claims);

        return jwtEncoder
                .encode(parametros)
                .getTokenValue();
    }
}

```

Rota de login

Conferindo as credenciais e emitindo o JWT

No pacote controller, crie AuthController.java com a rota:

- POST /auth/login

O AuthenticationManager recebe login e senha. O Spring utiliza AutenticacaoService, UsuarioRepository e BCrypt para conferir as credenciais.

Se estiverem corretas, o TokenService gera o JWT. Se estiverem incorretas, a API responde 401 Unauthorized.

AuthController.java

```java
package br.com.senai.teste.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.senai.teste.dto.LoginRequest;
import br.com.senai.teste.dto.TokenResponse;
import br.com.senai.teste.service.TokenService;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final TokenService tokenService;

    public AuthController(
            AuthenticationManager authenticationManager,
            TokenService tokenService) {

        this.authenticationManager = authenticationManager;
        this.tokenService = tokenService;
    }

    @PostMapping("/login")
    public ResponseEntity<TokenResponse> login(
            @Valid @RequestBody LoginRequest dados) {

        try {
            UsernamePasswordAuthenticationToken credenciais =
                    new UsernamePasswordAuthenticationToken(
                            dados.getLogin(),
                            dados.getSenha()
                    );

            Authentication autenticacao =
                    authenticationManager.authenticate(credenciais);

            String token = tokenService
                    .gerarToken(autenticacao.getName());

            return ResponseEntity.ok(
                    new TokenResponse(token)
            );

        } catch (AuthenticationException erro) {
            return ResponseEntity
                    .status(HttpStatus.UNAUTHORIZED)
                    .build();
        }
    }
}
```

Rotas publicas e protegidas

Finalizando o SecurityFilterChain

O SecurityFilterChain libera somente o cadastro e o login. Todas as outras rotas exigem autenticação.

O oauth2ResourceServer().jwt() informa que a API recebera JWT no formato Bearer Token. O Spring extrai o token do cabeçalho e utiliza o JwtDecoder para valida-lo.

- - POST /usuarios: publica;
- - POST /auth/login: publica;
- - demais rotas: JWT obrigatório;
- - sessao: desativada com STATELESS.

SecurityConfig.java

```java
@Bean
public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

    http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                    .sessionCreationPolicy(
                            SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                    .requestMatchers(
                            HttpMethod.POST,
                            "/usuarios",
                            "/auth/login")
                    .permitAll()
                    .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2
                    .jwt(Customizer.withDefaults()));

    return http.build();
}
```

Primeiro teste no Postman

Cadastrando um usuário

Envie:

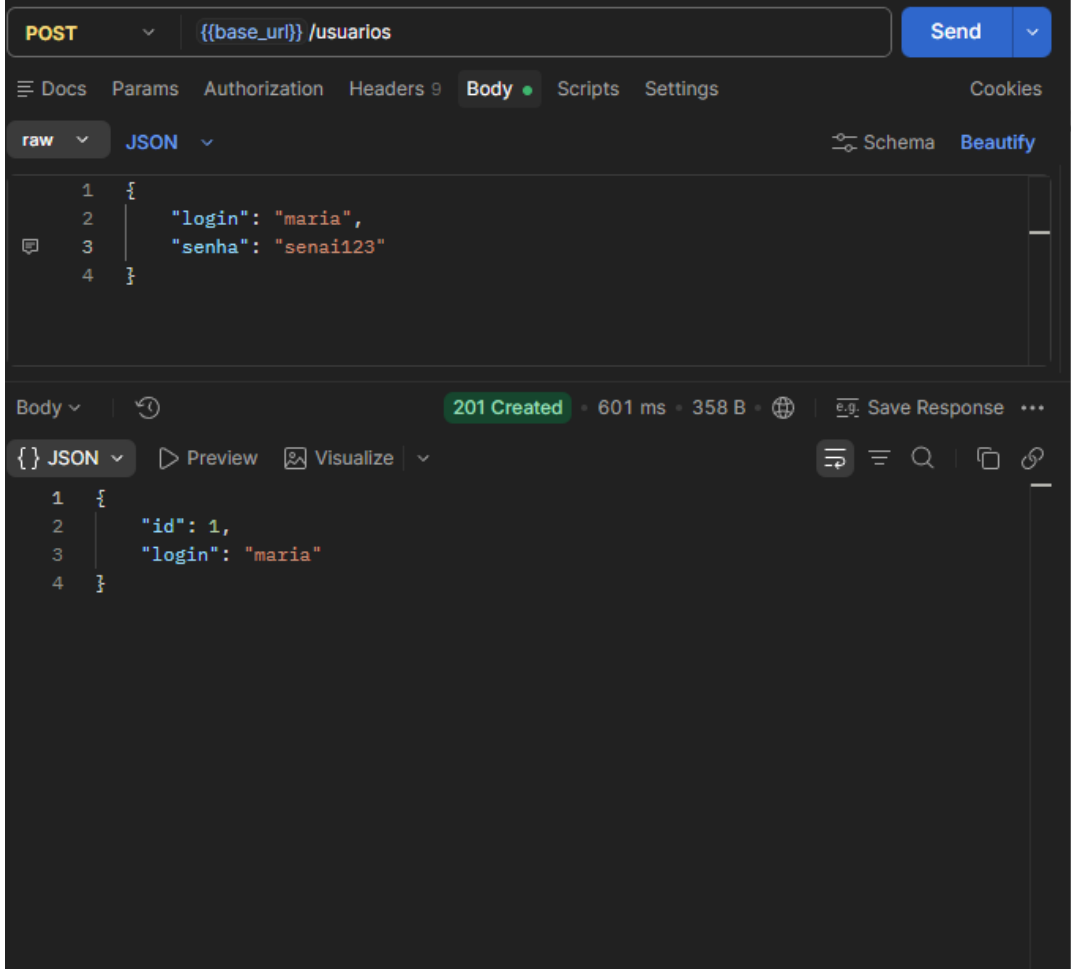
- http://localhost:8080/usuarios

Body > raw > JSON:

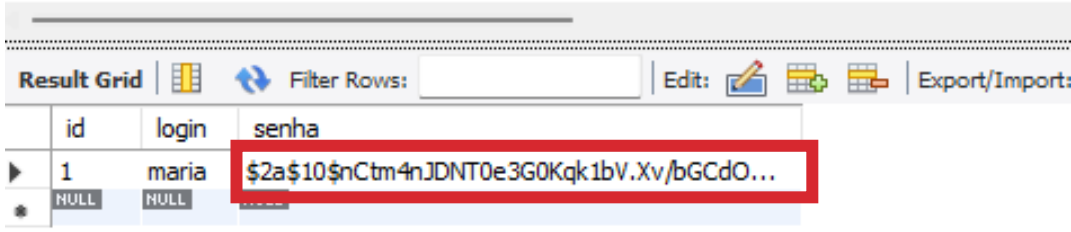
-

```
{
  "login": "maria",
  "senha": "senai123"
}
```

Confirme no banco que a senha foi armazenada como hash.







Obtendo o JWT

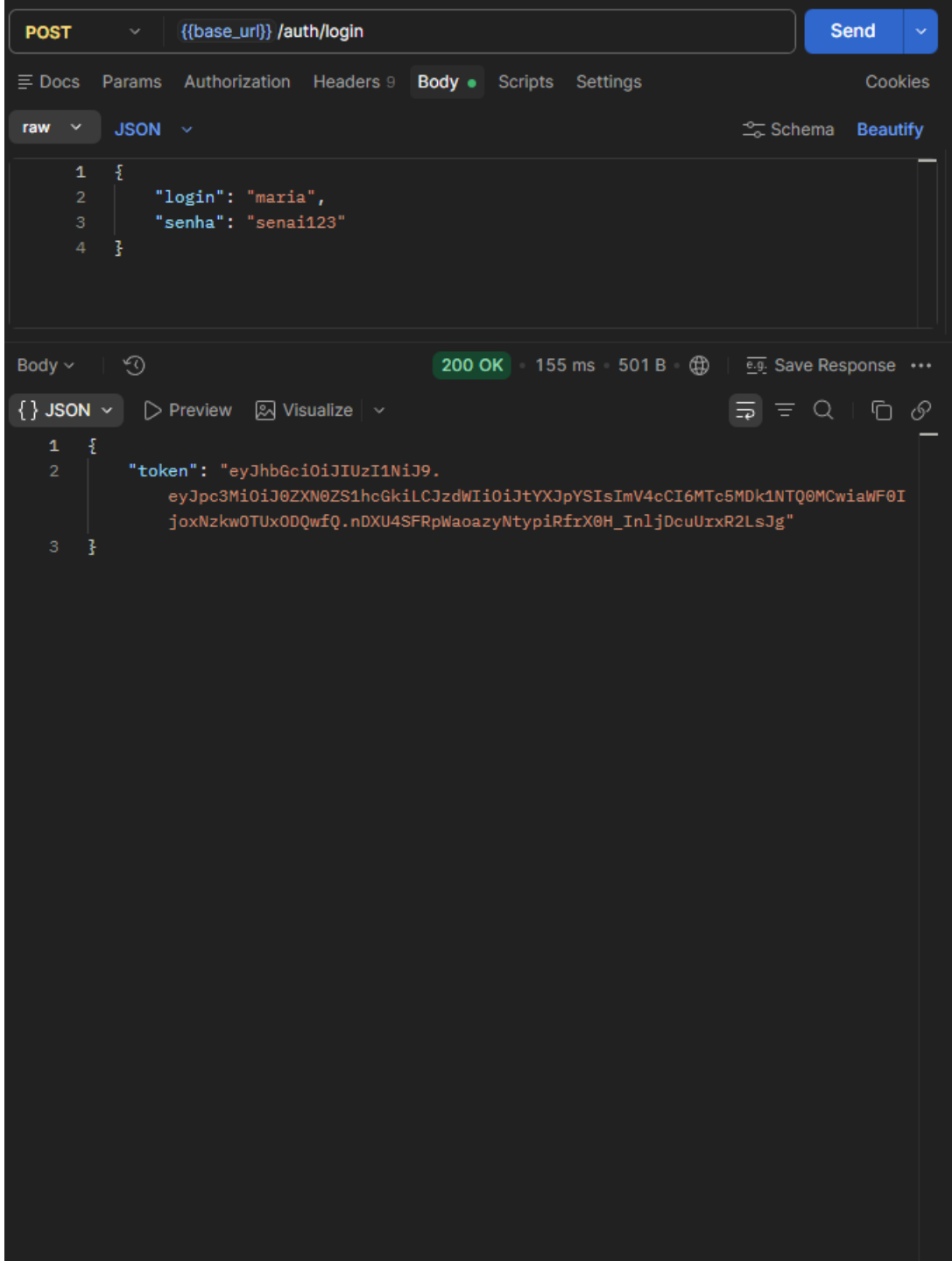
Autenticando o usuário

Envie:

- POST http://localhost:8080/auth/login

Body > raw > JSON:

-
```
{
  "login": "maria",
  "senha": "senai123"
}
```



Com as credenciais corretas, a API respondera `200 OK` e apresentara o token. Copie somente o valor existente dentro do campo `token`.

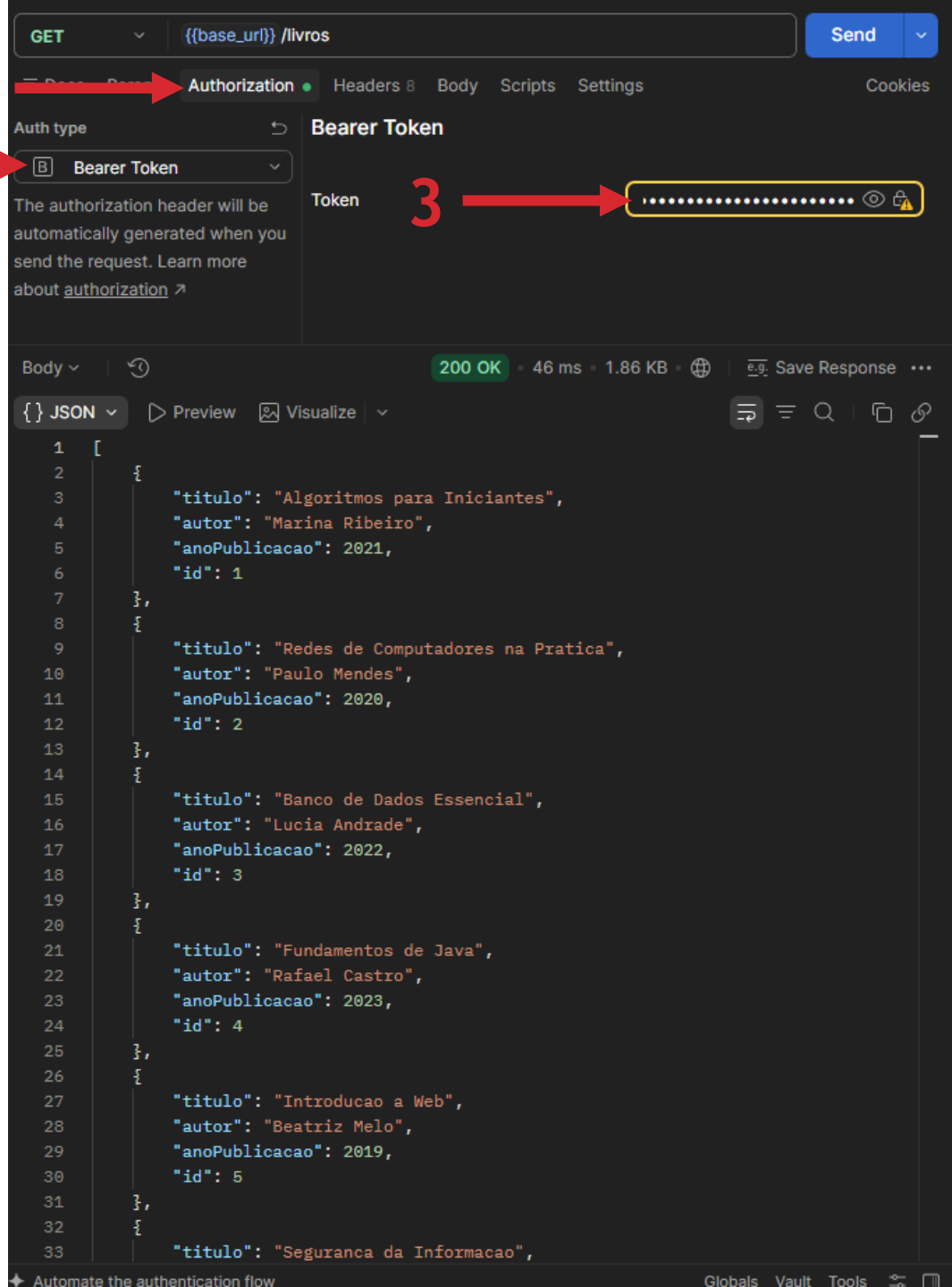
Com login inexistente ou senha incorreta, o resultado devera ser `401 Unauthorized`.

Enviando o Bearer Token

Acessando uma rota protegida

No Postman, vá para a rota:

- GET http://localhost:8080/livros
- Abra `Authorization`, selecione `Bearer Token` e cole o JWT recebido no login. Depois, envie a requisição.
- Com um token valido, o resultado será `200 OK`. Sem token, com token alterado ou expirado, a API respondera `401 Unauthorized`.



Firjan SENAI